
Neural Networks

Assignment #3

Synthetic Data Generation with VAE & GAN

Prepared by:

Name	ID
Zeyad Antar Othman Abu-Zaid	9220331
Abdelrahman Hatem Nasr Youssef	9220461
Abdallah Essam Mohamed Mahmoud	9220479

Submitted to:

Dr. Mohsen Rashwan
Dr. Mohamed Abdelghani

GitHub: [**GitHub Repository**](#)

May 2026

Contents

1	Introduction	2
2	Problem 1 — Synthetic Data Generation with Conditional VAE	2
2.1	Methodology	2
2.2	Experimental Setup	4
2.3	Results	5
2.4	Analysis	5
3	Problem 2 — Synthetic Data Generation with Conditional DCGAN	5
3.1	Methodology	5
3.2	Experimental Setup	6
3.3	Results	7
3.4	Analysis	7
3.5	VAE vs. GAN Comparison	7
4	Comprehensive Summary and Comparison with Assignment 2	8
4.1	Which GAN Selection Strategy Gives the Best Improvement?	8
4.2	Does Synthetic Data Reduce the Need for Real Data?	8
4.3	Comparison with Traditional Augmentation (Assignment 2)	8
4.4	Key Takeaways	10

1 Introduction

Deep learning models need a lot of data to work well, but getting large labelled datasets is often expensive or just not possible. In this assignment we look at ways to boost model performance when training data is limited.

For **Problems 1 and 2** we focus on **synthetic data generation** using the ReducedMNIST dataset (10 digit classes, 28×28 images, only **350 training samples per class**). We train a **Conditional VAE (CVAE)** and a **Conditional DCGAN (cDCGAN)**, use them to generate new images, filter those images by how confident a classifier is about them, and then check whether adding them actually helps classification.

2 Problem 1 — Synthetic Data Generation with Conditional VAE

2.1 Methodology

2.1.1 Data Loading and Preparation

The **ReducedMNIST** dataset has folders 0/ through 9/, each holding grayscale JPEG images of that digit. Although the full training set contains 1 000 images per class, we only use the **first 350 per class** on purpose to simulate having limited data. Every image is resized to 28×28 and normalised to $[0, 1]$.

2.1.2 Data Augmentation

With only 350 images per class there is a real risk of poor test accuracy. To help with this we apply **15× data augmentation** through simple geometric transforms:

- **Random rotation:** up to $\pm 15^\circ$
- **Random translation:** up to 10% horizontally and vertically
- **Random scaling:** between 90% and 110% of original size

These transforms keep each digit recognisable while adding variety similar to real handwriting differences. The training set grows from $350 \times 10 = 3,500$ to roughly $5,600 \times 10 = 56,000$ samples.

2.1.3 CVAE Architecture

A VAE learns a smooth latent space that we can sample from to create new images. By giving it the class label as an extra input, we get to choose *which digit* gets generated.

- **Encoder:** The input image (1, 28, 28) is concatenated with a learnable *label map* — a 28×28 embedding of the class label — along the channel dimension, yielding a (2, 28, 28) tensor. Three convolutional layers (with BatchNorm and ReLU) downsample this to a 2048-dimensional feature vector, which is projected to the mean μ and log-variance $\log \sigma^2$ of a 64-dimensional latent Gaussian.
- **Reparameterization:** A latent code $\mathbf{z}$ is sampled via $\mathbf{z} = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- **Decoder:** The latent code $\mathbf{z}$ is concatenated with a separate label embedding and projected back to (128, 4, 4). Three transposed-convolution layers upsample this to the reconstructed image (1, 28, 28) with a final Sigmoid activation.

2.1.4 Loss Function

The CVAE is trained to minimise:

$$\mathcal{L}_{\text{CVAE}} = \underbrace{\text{BCE}(\hat{\mathbf{x}}, \mathbf{x})}_{\text{reconstruction}} + \underbrace{D_{\text{KL}}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\text{regularisation}}$$

The **BCE term** pushes the output to look like the input, while the **KL term** keeps the latent distribution close to a standard normal so that we can smoothly interpolate between digits and sample meaningful new ones.

2.1.5 LeNet-5 Classifier

We use LeNet-5 for two things:

1. **Confidence filtering:** we first train it on the 350 real samples and use its softmax scores to judge how good each generated image looks.
2. **Final evaluation:** we then retrain it from scratch on real + synthetic data and measure test accuracy.

Architecture: Conv(1→6, 5×5) → ReLU → MaxPool → Conv(6→16, 5×5) → ReLU → MaxPool → FC(400→120) → FC(120→84) → FC(84→10). Trained for 20 epochs with Adam ($\text{lr} = 10^{-3}$).

2.1.6 Generation and Filtering Pipeline

1. **Generate:** 5 independent runs $\times$ 1 000 samples per class = 50 000 total.

2. **Score:** the pre-trained LeNet-5 computes the maximum softmax probability (confidence) for each generated image.
3. **Filter into three datasets:**
 - **Dataset A:** all 50 000 generated samples (no filtering).
 - **Dataset B:** only samples with confidence ≥ 0.9 (high quality).
 - **Dataset C:** samples with $0.6 \leq \text{confidence} < 0.9$ (medium quality).
4. **Retrain:** for each dataset, combine the 350 real samples with the synthetic subset and retrain LeNet-5 from scratch. Evaluate on the test set.

2.2 Experimental Setup

Parameter	Value
Dataset	ReducedMNIST (28×28 , 10 classes)
Real training data	350 images/class (limited)
Test data	200 images/class (2 000 total)
Augmentation	15× (rotation, translation, scaling)
CVAE latent dim	64
CVAE epochs	50
CVAE batch size	128
CVAE optimiser	Adam, lr = 10^{-3}
LeNet-5 epochs	20
LeNet-5 optimiser	Adam, lr = 10^{-3}
LeNet-5 batch size	64
Generation	5 runs $\times$ 1 000/class = 50 000
Confidence filter	LeNet-5 max-softmax

Table 1. Problem 1 hyperparameters (CVAE).

2.3 Results

Training Data	Synthetic Samples	Test Accuracy (%)
350 real (baseline)	0	95.90
350 real + VAE Dataset A (all)	50 000	97.30
350 real + VAE Dataset B (≥ 0.9)	32 968	96.85
350 real + VAE Dataset C (0.6–0.9)	11 821	98.45
1 000 real (upper baseline)	0	97.60

Table 2. LeNet-5 accuracy when augmented with CVAE-generated samples.

2.4 Analysis

Every synthetic dataset beats the 350-sample baseline of 95.90%. **Dataset C** (medium confidence, $0.6 \leq \text{conf} < 0.9$) does the best at **98.45%**, which is even higher than the 1 000-real upper baseline (97.60%).

Dataset B (≥ 0.9 confidence) only reaches 96.85%. It seems like the highest-confidence VAE samples are “too safe” — they look a lot like what the model already saw during training, so they do not really teach it anything new.

Dataset C has more variety: slightly unusual stroke styles, borderline shapes, and so on. That extra diversity helps the classifier generalise, almost like a form of regularisation.

The main takeaway here is that medium-confidence filtering works better than both no filtering and strict high-confidence filtering, at least for VAE-generated data.

3 Problem 2 — Synthetic Data Generation with Conditional DCGAN

3.1 Methodology

Everything from data loading to augmentation to the LeNet-5 pipeline stays the same as in Problem 1. The only difference is that we swap the CVAE for a **Conditional Deep Convolutional GAN (cDCGAN)**.

3.1.1 Generator Architecture

1. A 100-dimensional noise vector $\mathbf{z}$ is concatenated with a 10-dimensional label embedding.
2. A fully connected layer projects the 110-dimensional input to $256 \times 7 \times 7$ (with BatchNorm + ReLU).
3. Two transposed convolution layers upsample to $128 \times 14 \times 14$ and then $64 \times 28 \times 28$.
4. A final 3×3 convolution with Sigmoid produces the $(1, 28, 28)$ output.

3.1.2 Discriminator Architecture

1. The input image $(1, 28, 28)$ is concatenated with a label map $(1, 28, 28)$ along the channel dimension.
2. Two convolutional layers (with LeakyReLU and Dropout) downsample to $128 \times 7 \times 7$.
3. A fully connected layer outputs a single real/fake probability (Sigmoid).

3.1.3 Training Details

- **Loss:** Binary Cross-Entropy (BCE).
- **Label smoothing:** Real targets set to 0.9 instead of 1.0 to stabilise training.
- **Optimiser:** Adam with $\text{lr} = 2 \times 10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$ (standard DCGAN hyperparameters).
- **Epochs:** 100.

3.2 Experimental Setup

Parameter	Value
cDCGAN noise dim	100
cDCGAN epochs	100
cDCGAN batch size	128
cDCGAN optimiser	Adam, $\text{lr} = 2 \times 10^{-4}$, $\beta = (0.5, 0.999)$
Label smoothing	0.9
Generation	5 runs $\times$ 1 000/class = 50 000
Confidence filter	LeNet-5 max-softmax

Table 3. Problem 2 hyperparameters (cDCGAN). Data, augmentation, and LeNet-5 settings are the same as Table 1.

3.3 Results

Training Data	Synthetic Samples	Test Accuracy (%)
350 real (baseline)	0	95.90
350 real + GAN Dataset A (all)	50 000	97.45
350 real + GAN Dataset B (≥ 0.9)	35 512	97.45
350 real + GAN Dataset C (0.6–0.9)	10 120	97.55
1 000 real (upper baseline)	0	97.25

Table 4. LeNet-5 accuracy when augmented with cDCGAN-generated samples.

3.4 Analysis

All three GAN-augmented setups beat the 350-sample baseline. What stands out is how close the numbers are: Datasets A, B, and C give 97.45%, 97.45%, and 97.55%. The filtering strategy barely matters here, which tells us the cDCGAN is producing pretty good samples across the board.

Another thing worth noting is that the GAN ended up with a bigger Dataset B (35 512 vs. 32 968 for the VAE). GAN outputs tend to be *sharper*, so the classifier is more confident about them on average.

3.5 VAE vs. GAN Comparison

Configuration	VAE Acc. (%)	VAE Samples	GAN Acc. (%)	GAN Samples
350 real baseline	95.90	—	95.90	—
+ Dataset A (all generated)	97.30	50 000	97.45	50 000
+ Dataset B (conf ≥ 0.9)	96.85	32 968	97.45	35 512
+ Dataset C ($0.6 \leq \text{conf} < 0.9$)	98.45	11 821	97.55	10 120
1 000 real baseline	97.60	—	97.25	—

Table 5. Side-by-side comparison of VAE vs. GAN synthetic augmentation.

- **VAEs** are trained with a reconstruction + KL loss, which gives them a smooth latent space. The downside is that their outputs tend to be a bit *blurry*. Still, the diversity in Dataset C

turned out to be very helpful, giving us the best overall accuracy (**98.45%**).

- **GANs** learn by competing (generator vs. discriminator) and produce *sharper* images. Their numbers are steady across all filtering levels (97.45–97.55%), so they are a safe bet if you do not want to worry about picking the right confidence threshold.
- **Best overall:** VAE Dataset C at 98.45%. The slightly blurry but diverse samples from the CVAE end up being the most useful for training.
- **Both models beat the 350-sample baseline** and get close to (or pass) the 1 000-real upper bound, which shows that generating synthetic data really does work when you do not have enough real images.

4 Comprehensive Summary and Comparison with Assignment 2

4.1 Which GAN Selection Strategy Gives the Best Improvement?

Out of the three filtering strategies, **Dataset C** (medium confidence, $0.6 \leq \text{conf} < 0.9$) gives the best GAN accuracy at **97.55%**, just slightly above Datasets A and B (both 97.45%). For the CVAE the gap is bigger — Dataset C hits **98.45%**, the best number we saw anywhere.

Why does medium confidence win? Those samples are different enough to teach the classifier something new, but still look enough like real digits to be useful. The high-confidence ones (Dataset B) are basically copies of what the model already knows, and the unfiltered set (Dataset A) has some bad samples mixed in.

4.2 Does Synthetic Data Reduce the Need for Real Data?

Yes. Starting from just **350 real images per class**, adding either VAE or GAN samples pushes accuracy past the 350-real baseline (95.90%) and even past the **1 000-real upper baseline** (97.25–97.60%). VAE Dataset C gets to 98.45%, which we probably could not reach even with 1 000 real images. So if collecting more real data is hard or expensive, generating synthetic samples and filtering them carefully is a solid alternative.

4.3 Comparison with Traditional Augmentation (Assignment 2)

In Assignment 2 we tested traditional augmentation (rotation, translation, Gaussian noise) and also plain GAN synthesis on the same ReducedMNIST dataset with LeNet-5. The table below puts all the important numbers from both assignments side by side.

Source	Configuration (350 real/class base)	Test Acc. (%)
<i>Baselines</i>		
Asgn. 2	350 real only	94.90–96.35
Asgn. 3	350 real only	95.90
Asgn. 2	1 000 real only	96.90–97.30
Asgn. 3	1 000 real only	97.25–97.60
<i>Assignment 2 — Traditional Augmentation (Problem 5)</i>		
Asgn. 2	350 real + 1 000 augmented/digit	96.80
Asgn. 2	350 real + 2 000 augmented/digit	97.20
<i>Assignment 2 — GAN Synthesis (Problem 6, no filtering)</i>		
Asgn. 2	350 real + 1 000 GAN synthetic/digit	95.80
Asgn. 2	350 real + 1 500 GAN synthetic/digit	96.60
<i>Assignment 2 — Combined Aug. + GAN (Problem 6.3)</i>		
Asgn. 2	350 real + 1 000 aug + 1 000 syn	97.75
<i>Assignment 3 — CVAE with Confidence Filtering (Problem 1)</i>		
Asgn. 3	350 real + VAE Dataset A (all, 50 000)	97.30
Asgn. 3	350 real + VAE Dataset B (≥ 0.9 , 32 968)	96.85
Asgn. 3	350 real + VAE Dataset C (0.6–0.9, 11 821)	98.45
<i>Assignment 3 — cDCGAN with Confidence Filtering (Problem 2)</i>		
Asgn. 3	350 real + GAN Dataset A (all, 50 000)	97.45
Asgn. 3	350 real + GAN Dataset B (≥ 0.9 , 35 512)	97.45
Asgn. 3	350 real + GAN Dataset C (0.6–0.9, 10 120)	97.55

Table 6. Consolidated results across Assignments 2 and 3 (all using 350 real images/class as the base training set).

4.4 Key Takeaways

1. **Filtering makes a big difference.** In Assignment 2 we just threw all GAN samples into training and got modest gains (95.80–96.60%). Here in Assignment 3, using confidence-based filtering on the same type of GAN bumps accuracy to 97.45–97.55%. The lesson is clear: it is not just about generating more data, it is about picking the *right* generated data.
2. **VAE Dataset C wins overall** at 98.45%, beating everything else including Assignment 2's best combined setup (97.75%). The CVAE's smooth latent space seems to produce just the right mix of variety and quality in the medium-confidence range.
3. **Plain augmentation still works really well.** Back in Assignment 2, simply rotating and shifting 350 real images gave 97.75% — no generative model needed. It is free, simple, and hard to beat, so it should always be the first thing to try.
4. **We can get by with less real data.** Several setups using only 350 real images per class beat the 1 000-real baseline. That means generative augmentation can cut the real data requirement by over 65% without hurting accuracy.